# TRACE

TRACKING, REPORTING, ANALYSIS & COMMUNITY EVIDENCE

## TRACE

**Tracking, Reporting, Analysis & Community Evidence**

Community vehicle tracking platform for neighborhood safety chapters. Reporters submit sightings from their phones. Operators triage, dispatch, and build intelligence from a desktop console. Three-vault database architecture ensures reporter identities stay separate from operational data.

TRACE runs on Vercel (free tier) with a Neon PostgreSQL database (free tier). No server to manage.

### What you need

- A GitHub account
- A Vercel account (sign up with GitHub)
- A Neon account (free at neon.tech)

### Steps

1. **Fork the repo.** Click Fork on [github.com/duke-of-beans/TRACE](https://github.com/duke-of-beans/TRACE).

2. **Create a Neon database.** Sign in at [neon.tech](https://neon.tech), create a project, name the database `trace`. Copy the pooler connection string.

3. **Connect to Vercel.** Sign in at [vercel.com](https://vercel.com), click "Add New Project," import your fork. Under Environment Variables, add `DATABASE_URL` with your Neon pooler connection string. Deploy.

4. **Push the schema.** In your Neon dashboard, open the SQL Editor. Paste the contents of `migrations/0000_init.sql`, then `migrations/0001_dispatch.sql`, then `migrations/0002_photos-and-fixes.sql`. Run each one.

5. **Seed the database.** In the Neon SQL Editor, run this query to create your chapter and first operator:

```
-- Create chapter
INSERT INTO ops.chapters (id, name, slug, sunset_days)
VALUES (gen_random_uuid(), 'My Chapter', 'my-chapter', 90);

-- Create first operator (use the chapter ID from above)
INSERT INTO ops.reporters (id, chapter_id, callsign, status)
VALUES (gen_random_uuid(),
  (SELECT id FROM ops.chapters LIMIT 1),
  'YOUR-CALLSIGN', 'active');

-- Give them operator role
INSERT INTO ident.reporter_identities (id, reporter_id, role)
VALUES (gen_random_uuid(),
  (SELECT id FROM ops.reporters WHERE callsign = 'YOUR-CALLSIGN'),
  'operator');
```

Replace `YOUR-CALLSIGN` with the operator's callsign (uppercase, letters and numbers only).

6. **Lock it down.** In Vercel, add these environment variables and redeploy:

| Variable | Value | Purpose |
|---|---|---|
| `TRACE_DISABLE_DEV_LOGIN` | `true` | Prevents login by callsign alone |
| `TRACE_DISABLE_TEST_CODE` | `true` | Disables the demo invite code |

7. **Generate your first invite code.** Log in to the operator console at `your-app.vercel.app/operator/` using your callsign. Go to Admin > Reporters > Generate Invite Code. Give the code to your first reporter via Signal or in person.

8. **Share the reporter app.** Send reporters to `your-app.vercel.app`. On iPhone: Safari > Share > Add to Home Screen. On Android: Chrome > three dots > Add to Home Screen. It installs as a standalone app.

## For reporters

Open the app. Enter your invite code. You see a plate input and a map.

**Report mode.** Type a plate, describe the activity, drop the location, submit. Takes 15 seconds. The operator sees it immediately in their triage queue.

**Check Plate mode.** Type a plate to see if it is already in the database. If it matches, you can escalate to a full report with one tap.

**Map tab.** See active dispatch pins dropped by the operator. Tap a pin to respond. The operator sees your status (responding, on scene).

## For operators

The operator console is a desktop application at `/operator/`.

**Triage.** Incoming sightings appear in a queue. Each one shows whether the plate matches a known vehicle (MATCH badge) or is new (NEW PLATE). Confirm and dispatch, dismiss with feedback to the reporter, or add the vehicle to tracking.

**Intel Map.** All sightings on a satellite map. Right-click to drop a dispatch pin. Click any sighting marker to see details. Time playback shows sighting patterns hour by hour. Corridor overlays trace vehicle movement paths.

**Vehicles and Actors.** Dossier pages for tracked vehicles and persons of interest. Photos, suspicion levels, sighting history, identifier records.

**Admin.** Configure vehicle types, suspicion ladders, dispatch event types, actor identifiers. Generate reporter invite codes.

## Three-vault database

TRACE separates data into three PostgreSQL schemas. If someone gains access to the operational data (vehicles, sightings, actors), they cannot determine who submitted any report. Reporter real identities live in a separate schema with a separate encryption key.

| Schema | Contains | Access |
|---|---|---|
| `ops` | Vehicles, sightings, actors, dispatches, chapters | Full CRUD |
| `ident` | Reporter identities, auth tokens, sessions | Auth service only |
| `evidence` | Write-once evidence locker, hash chain | INSERT + SELECT only |

A full dump of the `ops` schema reveals zero real identities. By architecture, not policy.

## Stack

- **API:** Hono (TypeScript) on Vercel Serverless Functions
- **Database:** Neon PostgreSQL + Drizzle ORM
- **Reporter app:** Preact PWA (mobile-first, installable)
- **Operator console:** React SPA (desktop-first)
- **Maps:** Leaflet.js with Esri/CARTO/OSM tile layers
- **Auth:** Invite codes (no email required), TOTP for operators

## Project structure

```
src/
  api/            Hono route handlers
    auth/         Invite-code and dev-login auth
    sightings/    Sighting intake + plate matching
    vehicles/     Vehicle dossier CRUD
    actors/       Actor profile management
    dispatch/     Dispatch event lifecycle
    geo/          Heatmap, corridors, co-occurrence, temporal
    admin/        Chapter configuration + reporter invites
  db/
    schema/       Drizzle schema (three vaults)
    connection.ts Three connection pools
    seed.ts       Demo data
  services/       Plate lookup, geospatial, notifications

pwa/              Reporter PWA (Preact)
  src/pages/      Report, Map, History, Settings tabs
  src/components/ Onboarding, panic button, pin lock

operator/         Operator console (React)
  src/pages/      Dashboard, Triage, Intel, Vehicles, Actors, Admin, Security
  src/components/ Map, time slider, UX primitives

api/index.ts      Vercel serverless entry point
shared/design/    Design tokens and theme
```

## Authentication model

Reporters authenticate with a one-time invite code generated by an operator. No email, no phone number, no password. The invite code is given in person or via an encrypted channel (Signal). Once used, the code is invalidated.

Operators authenticate with a callsign and access code. The first operator is created during initial setup (see Quick Deploy step 5).

## Production hardening checklist

Before giving anyone access to your TRACE instance:

- Set `TRACE_DISABLE_DEV_LOGIN=true` in Vercel env vars
- Set `TRACE_DISABLE_TEST_CODE=true` in Vercel env vars
- Change the default operator callsign from `OPERATOR` to something unique
- Verify the operator console rejects non-operator logins (it does by default)
- All access is logged to the audit table

## Who can see what

| Role | Sees | Does not see |
|---|---|---|
| Reporter | Their own submissions, active dispatch pins, plate check results | Other reporters, operator actions, vehicle dossiers, actor profiles |
| Operator | All sightings (by callsign, not real name), vehicles, actors, dispatches | Reporter real identities, Vault B data |
| Database admin | All three schemas | Nothing is hidden from the database admin |

## Reporter safety

The reporter app includes a panic button (configurable in Settings) and a PIN lock. The operator can remotely suspend a reporter's device via the Security page, which sends a kill signal on the next status check.

## Prerequisites

- Node.js 22+
- PostgreSQL 15+ (local, Docker, or WSL)

## Setup

```
git clone https://github.com/duke-of-beans/TRACE.git
cd TRACE
npm install
cd pwa && npm install && cd ..
cd operator && npm install && cd ..
cp .env.example .env
# Edit .env with your local PostgreSQL credentials
```

## Database setup

```
# Create the database
psql -U postgres -c "CREATE DATABASE trace;"

# Run migrations
npx drizzle-kit migrate

# Seed demo data
npx tsx src/db/seed.ts
```

## Run

```
# API server (port 3100)
npm run dev

# Reporter PWA (port 5173)
cd pwa && npm run dev

# Operator console (port 5174)
cd operator && npm run dev
```

Demo credentials: operator callsign `OPERATOR`, reporter invite code `TEST-CODE`.

TRACE is a Progressive Web App. No app store needed.

**iPhone (Safari only)**

1. Open your TRACE URL in Safari
2. Tap the Share button (square with arrow)
3. Scroll down, tap "Add to Home Screen"
4. Tap "Add"

The app appears on the home screen with a standalone icon. It runs full-screen without Safari's address bar.

**Android (Chrome)**

1. Open your TRACE URL in Chrome
2. Tap the three-dot menu
3. Tap "Add to Home Screen" or "Install App"
4. Tap "Add"

**Desktop (Chrome/Edge)**

1. Open your TRACE URL
2. Click the install icon in the address bar (or three-dot menu > Install)

- [Chapter Setup Guide](docs/CHAPTER_SETUP.md) — complete deployment walkthrough for new chapters
- [Dispatch System Design](DISPATCH_DESIGN.md)
- [Voice Guide](VOICE_GUIDE.md)
- [Design System](DESIGN_SYSTEM.md)
- [Security Architecture](docs/SECURITY_ARCHITECTURE.md)
- [Security Edge Cases](docs/SECURITY_EDGE_CASES.md)

- [Requirements Synthesis](docs/REQUIREMENTS_SYNTHESIS.md)